Project Specification Document

Project Group Information

Group Members:

- Gaurav Bakale
- Yu-Tzu Li
- Rahul Sharma
- Karan Srinivas

Project Topic/Name

SplitZilla - Collaborative Expense Splitting Application

Problem Statement

Managing shared expenses among friends, roommates, or travel groups is often chaotic and leads to confusion about who owes whom. Traditional methods like spreadsheets or manual tracking are error-prone and time-consuming. Our Expense Splitting Application solves this problem by providing a centralized platform where users can create groups, add expenses, and automatically calculate balances using various splitting methods. The system simplifies debt settlement by tracking all transactions and notifying members of updates in real-time. This application ensures transparency, reduces conflicts, and makes shared expense management effortless for all users.

UML Diagram

```
┌──────────────────────────┐
│         User             │
├──────────────────────────┤
│ - userId: string         │
│ - name: string           │
│ - email: string          │
│ - password: string       │
├──────────────────────────┤
│ + register()             │
│ + login()                │
│ + joinGroup()            │
└──────────────────────────┘
            │
            │
            │ * belongs to
            │
            ▼
┌──────────────────────────┐
│         Group            │
├──────────────────────────┤
│ - groupId: string        │
```

```
    │ - name: string           │
    │ - description: string    │
    │ - members: List<User>    │
    │ - expenses: List<Exp>    │
    ├──────────────────────────┤
    │ + addMember()            │
    │ + removeMember()         │
    │ + addExpense()           │
    │ + getBalances()          │
    └──────────────────────────┘
                │
                │ contains *
                │
    ┌───────────▼──────────────┐
    │        Expense           │
    ├──────────────────────────┤
    │ - expenseId: string      │
    │ - description: string    │
    │ - amount: float          │
    │ - paidBy: User           │
    │ - date: datetime         │
    │ - splitStrategy: ISplit  │
    ├──────────────────────────┤
    │ + calculate()            │
    │ + notifyMembers()        │
    └──────────────────────────┘
                │
                │ uses
                │
    ┌───────────▼──────────────────────────────┐
    │      <<interface>> ISplitStrategy        │
    ├──────────────────────────────────────────┤
    │ + split(amount, members): Map<User, float>  │
    └──────────────────────────────────────────┘
                │
                │ implements
        ┌───────┴──────────────┬──────────────┐
        │                      │              │
    ┌───▼──────────┐   ┌───▼──────────┐  ┌───▼──────────┐
    │ EqualSplit   │   │ PercentSplit │  │ ExactSplit   │
    ├──────────────┤   ├──────────────┤  ├──────────────┤
    │ + split()    │   │ + split()    │  │ + split()    │
    └──────────────┘   └──────────────┘  └──────────────┘

    ┌──────────────────────────┐
    │  <<interface>> IObserver │
    ├──────────────────────────┤
    │ + update(event)          │
    └──────────────────────────┘
                │
                │ implements
    ┌───────────▼──────────────┐
    │  NotificationService     │
    ├──────────────────────────┤
```

```
│ - observers: List          │
├────────────────────────────┤
│ + attach(observer)         │
│ + detach(observer)         │
│ + notify(event)            │
└────────────────────────────┘

┌────────────────────────────┐
│   GroupFactory             │
├────────────────────────────┤
│ + createGroup(type)        │
└────────────────────────────┘

┌────────────────────────────┐
│   CommandManager           │
├────────────────────────────┤
│ - history: Stack           │
├────────────────────────────┤
│ + execute(command)         │
│ + undo()                   │
│ + redo()                   │
└────────────────────────────┘

┌────────────────────────────┐
│  DBConnectionManager       │
│     <<Singleton>>          │
├────────────────────────────┤
│ - instance: DBConn         │
├────────────────────────────┤
│ + getInstance()            │
│ + getConnection()          │
└────────────────────────────┘
```

Design Patterns to be Implemented

1. **Strategy Pattern**
   - **Purpose:** Implement different expense splitting algorithms (equal split, percentage split, exact amount split)
   - **Implementation:** `ISplitStrategy` interface with concrete implementations for each splitting method
2. **Observer Pattern**
   - **Purpose:** Notify group members when expenses are added, modified, or payments are made
   - **Implementation:** `IObserver` interface with `NotificationService` to manage and notify subscribers
3. **Factory Pattern**
   - **Purpose:** Create different types of groups or expense categories dynamically

- **Implementation:** `GroupFactory` class to instantiate various group types based on requirements

4. **Singleton Pattern**
   - **Purpose:** Ensure single instance of database connection manager or notification service
   - **Implementation:** `DBConnectionManager` class with private constructor and static instance
5. **Command Pattern**
   - **Purpose:** Enable undo/redo functionality for expense operations and maintain transaction history
   - **Implementation:** `CommandManager` with command objects for each operation (AddExpense, DeleteExpense, etc.)

Tech Stack

Backend

- **Framework:** Java + Spring Boot
- **Database:** PostgreSQL
- **ORM:** Spring Data JPA / Hibernate
- **Authentication:** JWT (JSON Web Tokens) with Spring Security
- **API Style:** RESTful API

Frontend

- **Framework:** React.js
- **UI Library:** Material-UI / Tailwind CSS
- **State Management:** React Context API / Redux
- **HTTP Client:** Axios

Additional Tools

- **Version Control:** Git & GitHub
- **API Documentation:** Swagger/OpenAPI (SpringDoc)
- **Testing:** JUnit 5 + Mockito (backend), Jest (frontend)
- **Build Tool:** Maven / Gradle

Functionalities to be Implemented by Milestone 2

1. **User Management System**
   - User registration with email validation
   - User login with JWT authentication
   - User profile management
2. **Group Management**

    - Create new groups with name and description
    - Add/remove members to/from groups
    - View all groups a user belongs to
    - View group details and member list

3. **Expense Management**
    - Add expenses with description, amount, and payer information
    - Implement 3 splitting strategies:
        - Equal split (divide equally among all members)
        - Percentage split (custom percentage for each member)
        - Exact amount split (specify exact amount per member)
    - View all expenses within a group
    - Delete expenses (with Command pattern for undo)
4. **Balance Calculation Engine**
    - Calculate individual balances (who owes whom)
    - Display simplified debt structure within groups
    - View personal balance summary across all groups
5. **Notification System (Observer Pattern)**
    - Real-time notifications when:
        - User is added to a group
        - New expense is created in user's group
        - User is involved in an expense split
6. **Transaction History**
    - View chronological list of all expenses in a group
    - Filter expenses by date, member, or amount
    - Export expense history (basic CSV export)

Contributions

Team Member	Contribution
Gaurav Bakale	Backend API development, Strategy pattern implementation for split methods
Yu-Tzu Li	Frontend React components, UI/UX design, user interface development
Rahul Sharma	Database schema design, Observer & Factory pattern implementation
Karan Srinivas	Authentication system, Command & Singleton pattern implementation

---

Project Timeline

- **Milestone 1:** Project specification and initial setup (Current)
- **Milestone 2:** Core functionalities implementation (as listed above)

- **Final Submission:** Complete application with all features, testing, and documentation

---

Document Version: 1.0
Last Updated: March 17, 2026